

Review SCSF và 3 hướng cải tiến risk-coverage trong training

Contents

1. Kết luận để lựa chọn hướng nghiên cứu	2
2. “Tăng accuracy” phải được hiểu như thế nào?	2
3. Baseline thực sự làm gì?	3
3.1. Kiến trúc và gradient theo PDF	3
3.2. Những điểm cần khóa lại khi tái lập	3
4. Điểm mạnh, điểm yếu và cơ hội của bài toán	4
4.1. Điểm mạnh của bài toán và SCSF	4
4.2. Điểm yếu quan trọng đối với risk-coverage	4
4.3. Các kết quả gốc chỉ ra vùng nào đáng cải tiến?	5
5. Nền tảng chung: tác động của một lỗi lên đường RC	6
6. Phương pháp 1 — R3-SCSF: phân bố giữa sửa lỗi và sửa thứ hạng	7
6.1. Giả thuyết và điểm khác baseline	7
6.2. Đo repairability bằng một bước cập nhật ảo	7
6.3. Objective cụ thể	7
6.4. Vì sao có thể cải thiện RC?	8
6.5. Thực nghiệm phân biệt contribution với tổ hợp loss thông thường	8
7. Phương pháp 2 — DTR-SCSF: học và sửa chuyển tiếp đúng→sai giữa các tầng	8
7.1. Giả thuyết và thí nghiệm chẩn đoán trước khi đầu tư	8
7.2. Confidence head học bốn trạng thái chuyển tiếp	9
7.3. Repair bằng distillation chọn đúng trường hợp	9
7.4. Vì sao khác thêm deep supervision hoặc voting?	10
7.5. Novelty và đối chứng gần nhất	10
7.6. Ablation bắt buộc và tiêu chí loại	10
8. Phương pháp 3 — CBR-SCSF: tối ưu cập nhằm lẫn với ngân sách coverage từng lớp	11
8.1. Giả thuyết	11
8.2. Một score tạo các tập nhận lẫn nhau	11
8.3. Soft confusion risk trên tập được nhận	11
8.4. Coverage floor và objective	12
8.5. Novelty nằm ở đâu, và giới hạn nào cần thừa nhận?	13
8.6. Ablation và điều kiện giữ hướng	13
9. Ma trận novelty và đối chứng tối thiểu	13
10. Protocol thực nghiệm để chứng minh cải tiến là thật	14
10.1. Khóa baseline trước	14
10.2. Chia dữ liệu và ngân sách công bằng	14
10.3. Bộ dữ liệu theo thứ tự đầu tư	15
10.4. AURC, E-AURC và threshold phải nhất quán	15
10.5. Phân biệt gain classifier, gain score và gain cơ chế	15

11. Điểm thay đổi dự kiến trong source khi triển khai	16
12. Đề xuất chốt hướng sau vòng thí nghiệm đầu	16

Ngày review: 10/09/2026.

Baseline: *SCSF: Selective Classification with Supervised Features for Medical Imaging*, bản PDF 17 trang do người dùng cung cấp: [ECCV_2026_SCSF \(1\).pdf](#).

Mục tiêu: thiết kế ba phương pháp có thể tăng **selective accuracy tại cùng coverage**, giảm AURC và có cơ chế cải tiến trong training đủ cụ thể để nghiên cứu thành contribution.

Đây là đề xuất nghiên cứu, chưa phải kết quả thực nghiệm. Không thể bảo đảm trước rằng một phương pháp sẽ tăng accuracy hoặc được công nhận là novel. Báo cáo phân biệt rõ: dữ kiện từ baseline, phát hiện từ source code, suy luận toán học, và giả thuyết đề xuất. Nội dung PDF được dùng làm tài liệu phân tích, không được coi là chỉ dẫn thay thế yêu cầu của người dùng.

1. Kết luận để lựa chọn hướng nghiên cứu

Hướng tối ưu tiên là phương pháp 2, nếu phép đo ban đầu xác nhận có đủ lỗi “tăng giữa đúng, tăng cuối sai”. Hướng này bám trực tiếp giả thuyết kiến trúc của SCSF, có đường tác động đến cả classifier và confidence, và có thể kiểm chứng cơ chế bằng những nhóm lỗi cụ thể.

Phương pháp đề xuất	Cơ chế mới cần kiểm chứng	Tác động mong muốn lên RC	Đánh giá ban đầu
1. R3-SCSF — Risk-weighted Repair-or-Rank Training	Ước lượng khả năng sửa từng lỗi bằng bước cập nhật ảo, rồi phân bố gradient giữa sửa classifier và sửa thứ hạng theo tác động biên lên RC	Giảm lỗi tự tin cao; cải thiện accuracy khi lỗi còn sửa được	Trực tiếp về objective, nhưng overhead và chất lượng tín hiệu repairability là rủi ro lớn
2. DTR-SCSF — Depth-Transition Repair	Học trạng thái đúng/sai giữa tầng giữa và tầng cuối; distillation có điều kiện để sửa chuyển tiếp đúng→sai gây hại cho RC	Vừa sửa lỗi của backbone vừa nhận diện lỗi còn tồn tại	Ưu tiên nghiên cứu; phải vượt deep supervision và reverse distillation thông thường
3. CBR-SCSF — Confusion-Budgeted Risk Training	Tối ưu các cặp nhầm lẫn ở nhiều coverage bằng cùng một score, kèm coverage tối thiểu theo lớp trong training	Hạ lỗi nhầm lớp khó mà không đạt kết quả chỉ bằng loại gần hết lớp đó	Hợp dữ liệu y khoa mất cân bằng; có thể phải đánh đổi micro-accuracy

Các tên viết tắt trên là **tên làm việc trong tài liệu này**, không phải tên phương pháp đã được công bố. Ba hướng có thể triển khai độc lập từ SCSF. Không cần ghép cả ba để tạo một bài báo.

Không nên nhận làm novelty: đổi MSE thành BCE, thêm ranking loss đơn giản, thêm SoftRank-AURC, thêm vài auxiliary heads, dùng contrastive loss có confidence, hoặc đổi lịch λ . Những thành phần này đã có trong paper, source code hiện tại hoặc công trình liên quan.

2. “Tăng accuracy” phải được hiểu như thế nào?

Cho classifier f_θ , confidence $g_\phi(x)$, và ngưỡng h :

$$\hat{y}_i = \arg \max_k f_\theta(x_i)_k, \quad e_i = \mathbf{1}[\hat{y}_i \neq y_i], \quad a_i(h) = \mathbf{1}[g_\phi(x_i) \geq h].$$

$$\text{coverage}(h) = \frac{1}{n} \sum_i a_i(h), \quad R(h) = \frac{\sum_i a_i(h)e_i}{\sum_i a_i(h)}, \quad \text{Acc}_{sel}(h) = 1 - R(h).$$

Risk không xác định khi không có mẫu nào được nhận. Với coverage $c > 0$, đánh giá bằng top- k , $k = \max(1, \lfloor cn \rfloor)$, theo confidence giảm dần.

Cần tách hai mục tiêu:

1. **Selective accuracy tại coverage cố định:** ví dụ cùng nhận 95% số ảnh, phương pháp mới phải sai ít hơn. Đây là mục tiêu chính của yêu cầu.
2. **Full-coverage accuracy:** accuracy khi nhận 100% ảnh. Muốn tăng chỉ số này phải thay đổi dự đoán của classifier. Chỉ đổi score hoặc ngưỡng không làm accuracy 100% tăng.

Không được so accuracy ở 70% coverage của phương pháp mới với accuracy ở 95% coverage của baseline rồi kết luận tốt hơn. Một phép biến đổi đơn điệu tăng của score cũng không thay đổi RC nếu giữ thứ hạng và cách xử lý ties.

Metric chính nên đăng ký trước: AURC toàn miền; risk/accuracy tại 70%, 75%, 80%, 85%, 90%, 95%, 100%; partial AURC trong [0.8, 1]; coverage đạt được ở một mức risk cố định; full-coverage accuracy. Với dữ liệu mất cân bằng, thêm coverage và risk theo lớp tại **cùng ngưỡng toàn cục**.

3. Baseline thực sự làm gì?

3.1. Kiến trúc và gradient theo PDF

Theo §3.2–3.4, trang 6–10:

- Backbone giữ C class logits, không thêm một lớp reject.
- Lấy hai feature trung gian: pool4/pool5 với VGG16-BN; hai residual blocks cuối với ResNet-18.
- Pool feature về lưới nhỏ, flatten, ghép với logits và đưa vào MetaCalibrator MLP.
- **Logits đầu vào MetaCalibrator được stop-gradient; feature trung gian vẫn truyền gradient.**
- Nhãn confidence là correctness hiện tại, $t_i = 1 - e_i$, không phải TCP trong phần phương pháp của bản PDF này.

Viết lại objective với chỉ số lớp đúng được tường minh:

$$\mathcal{L}_{CE} = -\frac{1}{B} \sum_i \log p_\theta(y_i | x_i),$$

$$\mathcal{L}_{meta} = -\frac{1}{B} \sum_i w_i [t_i \log g_i + (1 - t_i) \log(1 - g_i)], \quad w_i = \begin{cases} \alpha_e & t_i = 0 \\ 1 & t_i = 1. \end{cases}$$

$$\mathcal{L}_{SCSF} = \mathcal{L}_{CE} + \lambda_t \mathcal{L}_{meta}.$$

t_i là nhãn rời rạc được tính lại theo classifier hiện tại, không backprop qua argmax. Meta-loss tác động vào feature chung nên có thể gián tiếp đổi logits và decision boundary, dù đường trực tiếp qua logits vào MetaCalibrator bị chặn.

PDF §4.1 nêu cosine decay $\lambda_{max} = 1$, $\lambda_{min} = 10^{-4}$, và $\alpha_e = 1$ **cho cả benchmark chuẩn lẫn y khoa**. Vì vậy kết quả chính không tự chứng minh lợi ích của asymmetric weighting $\alpha_e > 1$.

3.2. Những điểm cần khóa lại khi tái lập

Bản PDF là mốc khoa học cho review này. Source trong workspace được dùng để xác định cấu hình và tránh đề xuất lại thứ đã có; chưa thể mặc định mọi file đều tạo ra bằng kết quả trong PDF.

Hạng mục	Bảng chứng đọc trực tiếp	Hệ quả cho nghiên cứu
PDF và README khác nhau	PDF dùng correctness-NLL; README mô tả TCP-MSE và post-hoc/detach	Không chạy nguyên lệnh README rồi gọi đó là tái lập đúng PDF

Hạng mục	Bảng chứng đọc trực tiếp	Hệ quả cho nghiên cứu
Target trong code	<code>train_scsf.py</code> , khoảng dòng 1100: chọn TCP. --nll chỉ đổi dạng loss	Muốn NLL trên correctness cần kết hợp --brier --nll; tên --brier dễ gây hiểu nhầm
Gradient thực tế	<code>MetaCalibrator.forward</code> , dòng 520-525: chế độ mặc định ghép feature chưa detach với <code>logits.detach()</code>	Code mặc định có gradient vào feature , trái comment “detach all”. Bật --end-to-end còn mở gradient qua logits, khác PDF
Weight trong NLL	<code>nll_loss</code> , dòng 175, trả mean không nhận weights; nhánh NLL dòng 1145 không dùng weights đã tính	Với $\alpha_e = 1$ phù hợp unweighted NLL; thử $\alpha_e > 1$ bằng cờ hiện có chưa thực hiện Eq. (9)-(10)
AURC khác quy ước	<code>train_scsf.py:860</code> tích phân trapezoid từ $1/n$ tới 1; <code>eval_scsf_multi_trial.py:156</code> dùng lưới coverage thừa từ 10% tới 100%	Hai script không đo cùng diện tích; phải chuẩn hóa trước khi so gains
“Nhiều trials” trong script eval	Một checkpoint được nạp rồi đổi shuffle seed và chia lại tập test	Đây là biến thiên do tập con đánh giá, không phải nhiều lần train độc lập ; chưa đủ bằng chứng để kết luận paper dùng chính quy trình này
Theo dõi test lúc train	<code>train_scsf.py:1589</code> đánh giá full test mỗi epoch; cuối cùng lại đánh giá test subset	Việc in test không tự chứng minh đã chọn model theo test, nhưng thực nghiệm mới cần tách checkpoint selection khỏi test
Kiến trúc eval	Script multi-trial khai báo MLP nhỏ hơn MLP 1024→512→256→128 của script train	Kiểm tra kiến trúc checkpoint, không mặc định script eval tương thích
Biến thể đã có	<code>train_ds_scsf.py</code> , dòng 148-322 và 467 trở đi: SoftRank, AURC surrogate, pairwise hinge, mini-classifiers, fusion confidence	Không nhận các thành phần này là đóng góp mới

Ngoài ra, `aurc_surrogate_loss` trong `train_ds_scsf.py` chuẩn hóa rank thành $(\text{ranks}-1)/n$ rồi lại chia cho $n+1$, khác biểu thức $r_i/(n+1)$ ở docstring. Hàm dùng confidence-MSE làm base loss. Vì vậy nó chưa phải đối chứng đáng tin cho direct classifier-AURC optimization chỉ dựa vào tên hàm. Chính công thức/tái lập đúng là công việc kỹ thuật, không phải novelty.

Không tìm thấy supplement mang tên tài liệu [2] của paper trong danh sách file hiện có; các kết luận phụ thuộc supplement về gradient blocking, lịch λ và error weight chưa được xác minh.

4. Điểm mạnh, điểm yếu và cơ hội của bài toán

4.1. Điểm mạnh của bài toán và SCSF

Bài toán gắn với giới hạn xử lý thực tế. Khi chỉ có thể chuyển một tỷ lệ ảnh sang chuyên gia, risk tại coverage cao trở thành mục tiêu cụ thể. Một đường RC tốt cho phép chọn nhiều mức tải xử lý mà không cần đổi classifier.

SCSF tận dụng thông tin sẵn có trong một forward pass. Feature nhiều độ sâu có thể bổ sung thông tin mà terminal confidence không biểu diễn được. Kiến trúc giữ output phân lớp thông thường nên tương đối thuận lợi để tích hợp.

Mục tiêu correctness phù hợp trực tiếp với failure prediction. Với classifier cố định và mô hình đủ khả năng học, BCE đúng/sai có cơ sở thống kê hợp lý. Không nên phê bình baseline rằng “BCE hoàn toàn không liên quan AURC”.

Có bằng chứng thực nghiệm hữu ích nhưng không đồng đều. Bảng 4 cho `pool4||pool5||logits` đạt AURC 0.0057, tốt hơn `pool4||logits` 0.0066. Bảng 3 cho SCSF đúng đầu 5/8 dataset. Các con số này ủng hộ hướng tiếp cận, nhưng chưa tách riêng hoàn toàn tác động feature, số tham số và chất lượng classifier.

4.2. Điểm yếu quan trọng đối với risk-coverage

W1 — Objective pointwise thiếu thông tin vị trí trong thứ hạng ở chế độ hữu hạn dữ liệu. Hai lỗi có cùng confidence bị BCE xử lý giống nhau dù vị trí và vùng coverage mà chúng

ảnh hưởng khác nhau. Đây là động lực thiết kế finite-sample optimization tốt hơn, không phải phủ nhận rank-optimality của BCE ở population optimum với classifier cố định.

W2 — Chưa phân biệt lỗi nên sửa và lỗi hiện tại nên hạ thứ hạng. Meta-loss có thể học nhận diện một lỗi mà không sửa được quyết định phân lớp. Ở coverage 100%, mọi cải thiện phải đến từ classifier; ở 90-95%, khả năng sửa các lỗi còn được nhận cũng rất quan trọng.

W3 — Nhiều độ sâu được concat, nhưng cơ chế chuyển tiếp chưa được giám sát tường minh. “Tầng giữa đúng, tầng cuối sai” khác “mọi tầng đều sai”. SCSF không có target riêng phân biệt hai trạng thái này. Tuy nhiên, chưa được coi mọi disagreement là lỗi, hoặc mọi feature collapse là mất thông tin hữu ích.

W4 — Correctness thay đổi theo thời gian và dễ cạn lỗi train. Khi train accuracy gần 100%, MetaCalibrator ít thấy negative examples dù validation vẫn còn lỗi. Với nhãn nhiều, correctness theo nhãn quan sát cũng không đồng nhất với tính đúng về mặt ngữ nghĩa. Đây là rủi ro cần đo, không phải hiện tượng đã được chứng minh trên các checkpoint chưa có.

W5 — Micro-AURC có thể che giấu việc loại bỏ lớp khó. Nhận chủ yếu lớp dễ có thể cho điểm trung bình đẹp nhưng làm coverage lớp hiếm thấp. Hiện tượng selective classification khuếch đại chênh lệch nhóm đã được nghiên cứu; không thể gọi việc phát hiện rủi ro này là novelty mới. [Jones et al., ICLR 2021](#).

W6 — Calibration không đồng nhất ranking. Từ Eq. (11), đặt $p = P(t = 1 | x)$:

$$g^*(p) = \frac{p}{p + \alpha_e(1 - p)}, \quad \frac{dg^*}{dp} = \frac{\alpha_e}{[p + \alpha_e(1 - p)]^2} > 0.$$

Do đó, tại nghiệm lý tưởng và classifier cố định, tăng α_e chỉ đổi thang confidence, không tự cải thiện ranking/AURC. Gains thực tế nếu có phải đến từ thay đổi tối ưu hữu hạn, representation hoặc dự đoán. Ví dụ ngưỡng 0.5 trong paper không đủ để chứng minh gain tại cùng coverage.

W7 — Lập luận gradient cần đọc đúng biến. PDF Eq. (12) so gradient theo xác suất g . Với $g = \sigma(s)$ và mẫu sai:

$$\frac{\partial L_{NLL}}{\partial s} = \alpha_e g, \quad \frac{\partial L_{MSE}}{\partial s} = 2g^2(1 - g)$$

cho MSE không trọng số như phép so của paper. NLL tránh gradient biến mất khi mẫu sai rất tự tin; nhưng gradient theo logit confidence không phân kỳ. Không suy ra backbone gradient “nổ vô hạn” chỉ từ Eq. (12).

4.3. Các kết quả gốc chỉ ra vùng nào đáng cải tiến?

CIFAR-10, Bảng 2, đơn vị error %.

Coverage	SCSF	SAT	Cách hiểu
100%	6.03 ± 0.17	6.05 ± 0.20	Chênh lệch mean chỉ 0.02 điểm %, chưa đủ để nói accuracy vượt chắc chắn
95%	3.76 ± 0.13	3.37 ± 0.05	Vùng yếu rõ để nghiên cứu; khoảng cách mean là 0.39 điểm %
90%	2.08 ± 0.04	1.93 ± 0.09	Vẫn thua SAT 0.15 điểm %
85%	1.06 ± 0.04	1.15 ± 0.18	Có lợi thế mean, nhưng phải xem biến thiên
75%	0.33 ± 0.02	0.44 ± 0.03	Lợi thế ranking ở coverage thấp hơn

Ví dụ, risk 95% giảm từ 3.76% xuống 3.40% sẽ tăng selective accuracy từ 96.24% lên 96.60%, tức **0.36 điểm phần trăm**, tương đương giảm risk tương đối khoảng 9.6%. Đây chỉ là ví dụ diễn giải metric, không phải dự báo kết quả.

Y khoa, Bảng 3, đơn vị AURC %.

Dataset	SCSF	Baseline tốt nhất trong bảng	Nhận xét
Breast Ultrasound	8.439	DG: 8.914	SCSF tốt hơn khoảng 5.33% tương đối
Tuberculosis	0.059	SN: 0.117	Tốt hơn khoảng 49.57%
Brain Cancer	0.017	SAT: 0.037	Tốt hơn khoảng 54.05%; test nhỏ cần đặc biệt xem uncertainty
COVID-QU-Ex	1.425	SAT: 1.910	Tốt hơn khoảng 25.39%
Chest CT	15.471	SN: 20.163	Tốt hơn khoảng 23.27%, nhưng risk tổng thể còn cao
Alzheimer	0.234	SN: 0.172	SCSF kém hơn
Brain Tumor MRI	0.062	DG: 0.012	SCSF kém hơn; tỷ lệ tương đối nhạy vì mẫu số rất nhỏ
Malaria	0.241	SAT: 0.237	Gần hòa, SCSF hơi kém hơn

Kiểm tra độc lập con số 31.52%: lấy $100(AURC_{best\ baseline} - AURC_{SCSF}) / AURC_{best\ baseline}$ rồi trung bình trên **năm hàng SCSF thắng** thu được 31.5237%. Con số này khớp abstract. Đây là cách tái dựng từ bảng, chưa phải xác nhận tác giả đã tính đúng theo cách này. Không được diễn giải thành mức cải thiện trung bình trên cả tám dataset.

Một số điểm báo cáo cần làm rõ thêm:

- Bảng 1 có SARS-CoV-2 CT, nhưng Bảng 3 có Breast Ultrasound thay cho dataset đó.
- Bảng 2 ghi “3 trials”, chưa giải thích đầy đủ tính độc lập của training seeds trong nội dung đã đọc.
- SCSF và SCSF+SR ở coverage 100% có mean khác nhau, rõ hơn ở Dogs vs. Cats. Nếu thực sự dùng cùng predictions và cùng test samples, đổi score không thể gây khác biệt này; cần xác nhận checkpoint/split/trials.
- Trên y khoa không có đối chứng SR trong bảng chính. Vì AURC phụ thuộc cả classification error lẫn ranking, cần thêm accuracy và E-AURC để giải thích nguồn gain.
- Diễn giải “giữ ambiguity” mới được hỗ trợ gián tiếp; cần đo lỗi theo độ sâu hoặc ablation có kiểm soát để kiểm tra cơ chế.

5. Nền tảng chung: tác động của một lỗi lên đường RC

Sắp mẫu theo score giảm dần; $r_i = 1$ là vị trí tự tin nhất. Chọn quy ước thực nghiệm:

$$AURC_n = \frac{1}{n} \sum_{k=1}^n \frac{1}{k} \sum_{j=1}^k e_{(j)} = \sum_{i=1}^n e_i W_n(r_i), \quad W_n(r) = \frac{1}{n} \sum_{k=r}^n \frac{1}{k}.$$

Đây là đẳng thức do đổi thứ tự tổng, **không phải định lý mới được nhận làm contribution**. Hai hệ quả hữu ích khi thiết kế training:

- Nếu sửa một lỗi ở hạng r thành đúng và giữ nguyên thứ hạng, AURC giảm đúng $W_n(r)$.
- Nếu đổi chỗ một mẫu sai ở r_i với mẫu đúng ở $r_j > r_i$, giữ các mẫu khác cố định, AURC giảm đúng

$$\Delta_{ij} = W_n(r_i) - W_n(r_j) = \frac{1}{n} \sum_{k=r_i}^{r_j-1} \frac{1}{k}.$$

Đối với partial RC hoặc lưới coverage ưu tiên, thay W bằng tổng chỉ trên các k được quan tâm với trọng số tích phân tương ứng. **Tối ưu partial RC không đồng nghĩa tối ưu AURC toàn miền.**

Trong training, n được thay bằng batch size hoặc tập tham chiếu train. Rank và W là trọng số stop-gradient; gradient hữu ích phải đi qua score loss hoặc classification loss. Không backprop trực tiếp qua hard 0-1 error rồi kỳ vọng classifier tự được tối ưu.

6. Phương pháp 1 — R3-SCSF: phân bổ giữa sửa lỗi và sửa thứ hạng

6.1. Giả thuyết và điểm khác baseline

Với một mẫu sai nhưng confidence cao, có hai can thiệp: làm classifier dự đoán đúng, hoặc làm score đưa nó xuống dưới những mẫu đúng. SCSF có CE và confidence loss nhưng chưa điều phối hai can thiệp theo **khả năng sửa lỗi hiện tại × tác động lên RC**.

Đề xuất đo một proxy cho khả năng sửa mẫu trong training bằng cập nhật ảo, rồi dùng nó để phân bổ **phần gradient bổ sung**. CE và BCE gốc vẫn được duy trì trên tất cả mẫu.

6.2. Đo repairability bằng một bước cập nhật ảo

Với mẫu train sai (x_i, y_i) , tạo hai view bảo toàn nhãn u_i, v_i . Gọi ω là tham số block cuối và classifier, còn các tham số khác giữ cố định trong phép thử:

$$\omega_i^+ = \omega - \eta_v \nabla_{\omega} CE(f_{\theta}(u_i), y_i).$$

Đo loss trên view còn lại trước và sau cập nhật:

$$\rho_i = \text{sg} \left[\text{clip} \left(\frac{CE(f_{\theta}(v_i), y_i) - CE(f_{\theta[\omega \leftarrow \omega_i^+]}(v_i), y_i)}{CE(f_{\theta}(v_i), y_i) + \epsilon}, 0, 1 \right) \right].$$

- ρ_i cao: cập nhật sửa lỗi trên một view cũng giúp view kia; tăng ưu tiên repair.
- ρ_i thấp: tăng ưu tiên ranking trong bước hiện tại.
- Tham số ảo được bỏ ngay sau khi đo. Không thay đổi optimizer state hoặc BatchNorm running statistics; cố định stochasticity khi so trước/sau.
- ρ_i **không phải xác suất lỗi bất khả khắc phục**, cũng không phải ước lượng generalization không chệch. Hai view vẫn cùng một ảnh; đặc biệt có thể cùng mang nhãn sai.

Bản thử đầu chỉ đo một tập nhỏ lỗi trong batch. Mẫu chưa đo nhận $\rho = 0.5$, hoặc dùng cache theo sample ID với hạn tuổi tối đa một epoch; không suy đoán chúng “không sửa được”.

6.3. Objective cụ thể

Đặt s_i là logit trước sigmoid của MetaCalibrator. $\bar{W}_i$ là $W_B(r_i)$ được chuẩn hóa về trung bình 1 trên tập lỗi và chặn giá trị cực lớn. Với cặp i sai, j đúng:

$$d_{ij} = \text{sg}(|W_B(r_i) - W_B(r_j)|), \quad \ell_{\text{rank}}(i, j) = T \log \left(1 + \exp \frac{m + s_i - s_j}{T} \right).$$

$$\mathcal{L}_{\text{rank}} = \frac{\sum_{i:e_i=1} \sum_{j:e_j=0} d_{ij}(\epsilon_r + 1 - \rho_i) \ell_{\text{rank}}(i, j)}{\sum_{i:e_i=1} \sum_{j:e_j=0} d_{ij} + \epsilon},$$

$$\mathcal{L}_{\text{repair}} = \frac{\sum_{i:e_i=1} \bar{W}_i \rho_i CE(f_{\theta}(x_i), y_i)}{\sum_i e_i + \epsilon},$$

$$\mathcal{L}_1 = \mathcal{L}_{SCSF} + \beta \mathcal{L}_{rank} + \gamma \mathcal{L}_{repair}.$$

Với cặp đang sai thứ tự, d_{ij} chính là độ giảm AURC khi đổi chỗ theo giả định §5. Với cặp đã đúng thứ tự, nó chỉ là trọng số bảo vệ margin, không phải một gain đã xảy ra. $\varepsilon_r > 0$ giữ tín hiệu ranking cả khi ρ_i cao. Batch thiếu mẫu đúng hoặc sai thì bỏ ranking term, không tạo NaN. Mẫu số không chuẩn hóa theo tổng repairability, để không triệt tiêu tác dụng phân bổ của gate khi mọi mẫu có cùng ρ thấp hoặc cao.

Đường gradient: repair cập nhật classifier và backbone; ranking cập nhật MetaCalibrator và feature trung gian. Logits đi vào MetaCalibrator vẫn stop-gradient như PDF. ρ_i , hard labels, hard ranks và trọng số đều detach; không cần meta-gradient bậc hai qua virtual step trong bản đầu.

6.4. Vì sao có thể cải thiện RC?

Lỗi xếp quá cao bị tăng trọng số theo ảnh hưởng của nó lên RC. Những lỗi có dấu hiệu sửa được nhận thêm CE, tạo đường tăng cả accuracy 100%. Những lỗi chưa sửa được nhận thêm ranking pressure. Đây là **cơ chế kỳ vọng**, không có định lý bảo đảm cập nhật SGD sẽ làm RC giảm vì rank và correctness của các mẫu khác cũng thay đổi.

Contribution có thể bảo vệ: một thuật toán điều phối sửa prediction/sửa ranking dựa trên đáp ứng với can thiệp training, có trọng số từ RC, và bằng chứng rằng chính sự điều phối tạo gain.

Phần đã có tiền lệ: ordinal confidence ranking đã có ở [CRL, ICML 2020](#); SoftRank cho AURC đã có trong [Zhou et al., bản v1 năm 2025](#); virtual-gradient example reweighting đã có ở [Ren et al., ICML 2018](#). [ReSiDe, 2026](#) còn kết hợp confidence nhiều tầng bằng preference optimization trong chế độ post-hoc. Vì vậy **“weighted ranking + SCSF” riêng lẻ chưa đủ**.

6.5. Thử nghiệm phân biệt contribution với tổ hợp loss thông thường

So sánh cùng backbone, số bước update và ngân sách tuning:

1. SCSF; SCSF + ranking đều trọng số; SCSF + RC-weighted ranking.
2. SCSF + thêm CE cho hard samples với ngân sách tương đương.
3. RC-weighted ranking + repair nhưng $\rho = 0.5$ cố định.
4. Đầy đủ; thay ρ bằng confidence, CE hiện tại, hoặc random cùng phân phối để kiểm tra giá trị của virtual probe.
5. Đầy đủ nhưng detach feature vào MetaCalibrator, để tách gain representation khỏi gain confidence head.

Đo thêm: ρ có dự báo được lỗi sẽ sửa ở các epoch sau không; high-confidence errors được chuyển thành đúng hay chỉ chuyển xuống rank thấp; gain có giữ ở risk@90/95 và accuracy@100 không.

Điều kiện loại hướng: nếu proxy không hơn CE/confidence đơn giản, hoặc gains chỉ do thêm compute/augmentation, phần novelty trọng tâm chưa được hỗ trợ. Overhead có thể đáng kể vì virtual update theo mẫu; bắt đầu với 4-8 lỗi mỗi 10 bước, đo wall-clock thay vì tuyên bố một tỷ lệ overhead chưa benchmark. Pairwise ranking thêm chi phí tối đa $O(B^2)$; inference giữ như SCSF.

7. Phương pháp 2 — DTR-SCSF: học và sửa chuyển tiếp đúng→sai giữa các tầng

7.1. Giả thuyết và thí nghiệm chẩn đoán trước khi đầu tư

SCSF cho rằng feature trung gian giữ tín hiệu bị mất ở các tầng sau. Cách kiểm tra cụ thể là gán một classifier phụ $q_\psi(y | h_4)$, đo mẫu nào **probe tầng giữa đúng nhưng classifier cuối sai** trên validation.

Trước tiên train probe với backbone đóng băng để đo chất lượng thông tin sẵn có; đây chỉ là phép chẩn đoán. Phương pháp chính phía dưới train đồng thời backbone, probe và confidence head. Cần xác nhận hiện tượng còn tồn tại trên held-out data, không chỉ trên train.

Không diễn giải một probe sai thành chứng minh representation đã mất thông tin: probe có thể thiếu capacity hoặc chưa được tối ưu. Ngược lại, probe đúng mà final sai là bằng chứng cụ thể hơn về một quyết định cuối chưa tận dụng được tín hiệu phân lớp sẵn có.

7.2. Confidence head học bốn trạng thái chuyển tiếp

Đặt:

$$t_i^S = \mathbf{1}[\arg \max q_\psi(x_i) = y_i], \quad t_i^L = \mathbf{1}[\arg \max p_\theta(x_i) = y_i].$$

Thay lớp output sigmoid một chiều của MetaCalibrator bằng softmax bốn chiều:

$$\pi_i^{ab} = Q_\phi(t^S = a, t^L = b \mid h_{4i}, h_{5i}, \text{sg}(z_i)), \quad a, b \in \{0, 1\}.$$

Trạng thái (t^S, t^L)	Ý nghĩa quan sát được	Tín hiệu training
(1,1)	Cả tầng giữa và cuối đúng	Giữ khả năng nhận mẫu
(0,1)	Tầng cuối sửa được quyết định sai của probe	Giữ lợi ích xử lý sâu; không ép copy probe
(1,0)	Quyết định đúng ở probe trở thành sai ở cuối	Ưu tiên repair có điều kiện
(0,0)	Cả hai đều sai theo nhãn quan sát	CE vẫn học; head học hạ confidence nếu chưa sửa được

Confidence cho **đự đoán cuối**, dùng cả train và inference:

$$g_i = \pi_i^{01} + \pi_i^{11}.$$

Điểm này tránh một lỗi thiết kế: không dùng “một tầng bất kỳ đúng” làm confidence cho final prediction. Mẫu (1,0) vẫn là lỗi của classifier cuối cho đến khi nó được sửa.

$$\mathcal{L}_{trans} = -\frac{1}{B} \sum_i \log \pi_i^{t_i^S, t_i^L}.$$

Q chỉ cần feature và logits như SCSF; probe sinh nhãn phụ và teacher trong training, **không bắt buộc chạy ở inference**. Không có ground-truth label hoặc phép kiểm tra correctness tại inference.

7.3. Repair bằng distillation chọn đúng trường hợp

Với hai view bảo toàn nhãn u_i, v_i , chỉ chọn mẫu mà probe đúng ở cả hai view, còn classifier cuối sai ở view chính:

$$d_i = \text{sg} [t_i^S(u_i) t_i^S(v_i) (1 - t_i^L(u_i))].$$

Đặt $q_i^T = \text{softmax}(z_i^S/T)$ và $p_i^T = \text{softmax}(z_i^L/T)$, cùng tính trên view chính. Dùng $W_B(r_i)$ của chính score DTR hiện tại, chuẩn hóa/chặn như §5:

$$\mathcal{L}_{DTR-repair} = \frac{\sum_i d_i \bar{W}_i T^2 D_{KL}(\text{sg}(q_i^T) \| p_i^T)}{\sum_i d_i + \epsilon}.$$

$$\mathcal{L}_2 = \mathcal{L}_{CE}^{final} + \beta \mathcal{L}_{CE}^{probe} + \lambda_t \mathcal{L}_{trans} + \gamma \mathcal{L}_{DTR-repair}.$$

CE final và probe vẫn dùng nhãn gốc. Teacher distribution, rank weights và state labels được detach. Distillation cập nhật final classifier/backbone, không đi ngược vào teacher qua nhánh KL; auxiliary CE vẫn cập nhật probe và feature của nó. Q vẫn có gradient vào feature trung gian và stop-gradient ở logits đầu vào.

Trong bản thử đầu dùng một probe tại pool4, không tăng số tầng ngay. Với ResNet-18 dùng layer3. Hai feature đưa vào Q vẫn là cặp feature của SCSF. Warmup cả final/probe trước khi bật transition loss và repair; điều chỉnh số epoch theo learning curves trên validation, không tự chọn bằng test.

7.4. Vì sao khác thêm deep supervision hoặc voting?

- Auxiliary CE đơn thuần yêu cầu probe dự đoán đúng; DTR còn dùng **trạng thái liên hệ giữa hai quyết định** để chọn can thiệp.
- Distillation mọi mẫu có thể kéo classifier cuối theo probe sai. DTR bảo vệ các trường hợp (0,1) bằng cách không distill từ probe ở đó.
- Q học likelihood của các chuyển tiếp, nhưng score cuối luôn marginalize theo **correctness của final classifier**.
- Trọng số RC tập trung repair vào lỗi (1,0) đang có confidence cao hoặc tác động lớn trên vùng coverage mục tiêu.

Một phân tích cần nói thẳng: joint NLL có thể viết thành

$$-\log P(t^S, t^L | v) = -\log P(t^L | v) - \log P(t^S | t^L, v).$$

Vì vậy đây là correctness supervision cộng một nhiệm vụ có điều kiện. Thêm softmax bốn trạng thái **không tự nó chứng minh cải thiện ranking**. Giá trị nghiên cứu nằm ở việc state supervision và can thiệp repair có điều kiện giải quyết được loại lỗi nào, với bằng chứng vượt auxiliary tasks có cùng capacity.

7.5. Novelty và đối chứng gần nhất

Hiện tượng destructive overthinking và disagreements giữa các tầng đã có ở **Shallow-Deep Networks, ICML 2019**. **TULIP, ICML 2024** dùng intermediate representations cho uncertainty estimation. Distillation từ shallow sang deep cũng đã xuất hiện trong **Multi-target Knowledge Distillation via Student Self-reflection, IJCV 2023**. Source train_ds_scsf.py đã có mini-classifiers.

Contribution dự kiến: *risk-weighted, transition-conditioned repair with final-correctness marginal confidence*. Cần chứng minh chuyển tiếp (1,0) là một nguồn gây RC xấu của SCSF; can thiệp đúng nhóm đó tạo gain mà deep supervision, disagreement scoring và reverse distillation đại trà không đạt được. Không nhận “overthinking”, “multi-head” hay “shallow-to-deep KD” là phát minh mới.

7.6. Ablation bắt buộc và tiêu chí loại

So sánh:

1. SCSF + một auxiliary CE, cùng số tham số và số view.
2. SCSF + reverse KD mọi mẫu; reverse KD chỉ khi probe đúng, chưa dùng điều kiện final sai.
3. DTR chỉ state head, chưa repair.
4. DTR repair đúng nhóm (1,0), chưa có RC weight.
5. DTR đầy đủ; thêm đối chứng nhãn phụ ngẫu nhiên/cân bằng có cùng số trạng thái để kiểm tra gain từ regularization đơn thuần.
6. DTR head so với score disagreement đơn giản và với SR trên cùng trained backbone.

Đo trên validation/test: tỷ lệ bốn trạng thái; tỷ trọng (1,0) trong lỗi được nhận ở 90%/95%; số mẫu (1,0) được sửa thành final đúng; accuracy@100; AURC và E-AURC. Đánh giá trạng thái theo cùng probe protocol và lưu predictions để làm paired comparison.

Loại hoặc hạ ưu tiên hướng nếu: số lỗi (1,0) trên held-out data quá ít; repair làm hỏng nhiều mẫu (0,1); hoặc gain biến mất khi so với auxiliary CE/reverse KD cùng budget. Nếu chỉ head bốn trạng thái tốt hơn nhưng repair không đóng góp, cần thu hẹp claim thành auxiliary supervision, không kể câu chuyện “sửa mất thông tin”.

Chi phí: thêm một probe và forward phụ cho kiểm tra view trong training. Q chỉ tăng output 1→4; probe có thể bỏ khi inference. Không cần ensemble hay test-time augmentation. Không có bằng chứng hiện tại để định lượng phần trăm tăng accuracy.

8. Phương pháp 3 — CBR-SCSF: tối ưu cặp nhằm lẫn với ngân sách coverage từng lớp

8.1. Giả thuyết

Trong dữ liệu có các lớp gần nhau về hình ảnh, lỗi nghiêm trọng đối với RC có thể tập trung ở một vài hướng nhằm $a \rightarrow b$, thay vì phân bố đều. Tối ưu loss trung bình để bỏ qua chúng. Ngược lại, nếu chỉ tăng trọng số lớp khó, selector có thể học reject phần lớn lớp đó.

Đề xuất cùng lúc: **tối ưu soft confusion risk của các mẫu được nhận ở nhiều coverage**, và **ràng buộc mức tiếp cận tối thiểu theo lớp trong training**, bằng một confidence score toàn cục. Đây là mục tiêu nghiên cứu cho phân lớp; không mặc định các lớp có mức độ hậu quả lâm sàng như nhau hoặc tự gán chi phí y khoa.

8.2. Một score tạo các tập nhận lồng nhau

Dùng score logit s_i của MetaCalibrator và lưới $\mathcal{C} = \{0.70, 0.80, 0.90, 0.95\}$. Với mỗi c :

$$a_{ic} = \sigma((s_i - h_c)/T_s), \quad \frac{1}{B} \sum_i a_{ic} = c.$$

h_c được giải để thỏa đẳng thức soft coverage, không phải một head riêng. Với cùng score và temperature, $c_1 < c_2$ dẫn tới $h_{c_1} > h_{c_2}$, do đó $a_{i,c_1} \leq a_{i,c_2}$. Khi inference, vẫn chỉ threshold một score như SCSF.

Phải dùng gradient của nghiệm quantile mềm, thay vì detach h_c rồi gọi đó là tối ưu fixed coverage. Với batch i.i.d.:

$$\frac{\partial h_c}{\partial s_j} = \frac{a_{jc}(1 - a_{jc})}{\sum_i a_{ic}(1 - a_{ic})}.$$

Có thể dùng implicit differentiation hoặc một differentiable sorting/top-k implementation được kiểm tra riêng. Công thức này suy ra bằng vi phân đẳng thức coverage; không nhận nó là novelty.

8.3. Soft confusion risk trên tập được nhận

Đặt $n_a = \sum_i \mathbf{1}[y_i = a]$. Coverage mềm của lớp thật a :

$$\phi_a(c) = \frac{\sum_{i:y_i=a} a_{ic}}{n_a}.$$

Với $a \neq b$, định nghĩa:

$$U_{ab}(c) = \frac{\sum_{i:y_i=a} a_{ic} p_\theta(b | x_i)}{\sum_{i:y_i=a} a_{ic} + \epsilon}.$$

Đây là **proxy mềm** cho khối lỗi $a \rightarrow b$ trong lớp a được nhận; không phải selective 0-1 error thực tế. Tổng theo $b \neq a$ là trung bình $1 - p_\theta(a | x)$ trên lớp a được nhận. Khi đánh giá, phải thay bằng hard predictions và hard acceptance.

Tại mỗi coverage, tập trung vào các cạnh nhầm khó bằng smooth maximum:

$$\mathcal{L}_{conf}(c) = \tau \log \left[\frac{1}{|\mathcal{E}|} \sum_{(a,b) \in \mathcal{E}} \exp \left(\frac{U_{ab}(c)}{\tau} \right) \right], \quad \mathcal{E} = \{(a,b) : a \neq b\}.$$

Tổng riêng theo từng coverage:

$$\mathcal{L}_{conf} = \sum_{c \in \mathcal{C}} \omega_c \mathcal{L}_{conf}(c), \quad \omega_c \geq 0, \quad \sum_c \omega_c = 1.$$

Không lấy một max duy nhất trên cả class-pairs lẫn coverage, vì nó dễ chỉ tập trung vào coverage cao nhất. Với nhiều lớp, chỉ giữ cạnh có đủ support theo thống kê train được cập nhật định kỳ; với bài toán 2-4 lớp có thể xét toàn bộ cạnh.

Thêm global soft selective risk để giữ mục tiêu trung bình:

$$\mathcal{L}_{micro} = \sum_c \omega_c \frac{\sum_i a_{ic} (1 - p_\theta(y_i | x_i))}{Bc}.$$

8.4. Coverage floor và objective

Áp đặt trong training:

$$\phi_a(c) \geq \kappa c, \quad 0 < \kappa \leq 1.$$

Ví dụ $\kappa = 0.8$, coverage toàn cục 90% tương ứng coverage mềm tối thiểu 72% cho mỗi lớp. Đây là thiết lập thử nghiệm, không phải chuẩn ứng dụng bắt buộc. Không đặt coverage floor cho “nhóm mẫu sai”, vì làm vậy có thể ép hệ thống nhận thêm lỗi.

Lagrangian:

$$\mathcal{L}_3 = \mathcal{L}_{SCSF} + \eta \mathcal{L}_{micro} + \beta \mathcal{L}_{conf} + \sum_{a,c} \nu_{ac} [\kappa c - \phi_a(c)], \quad \nu_{ac} \geq 0,$$

$$\nu_{ac} \leftarrow [\nu_{ac} + \eta_\nu (\kappa c - \phi_a(c))]_+.$$

Model cập nhật theo gradient descent, dual variables cập nhật theo ascent. Trong thực tế dùng EMA các residual, chặn dual và theo dõi độ vi phạm để tránh dao động; không tuyên bố hội tụ cho bài toán neural network không lỗi.

Đường gradient: p_θ trong soft confusion loss cập nhật classifier/backbone; a_{ic} cập nhật confidence head và intermediate features. Nhãn y chỉ dùng trong training. Đường logits vào confidence head giữ stop-gradient như SCSF.

Các công thức trên giả sử batch theo phân phối tự nhiên. Nếu class-balanced sampling, phải dùng importance weight theo tỷ lệ lớp thật trong các tổng global, cả phương trình coverage; nếu không sẽ vô tình tối ưu coverage của phân phối khác. Lớp vắng mặt không được tính risk bằng 0: bỏ term và không cập nhật dual của lớp đó ở bước ấy, hoặc dùng batch đủ support.

8.5. Novelty nằm ở đâu, và giới hạn nào cần thừa nhận?

Contribution dự kiến: training trên một tập các bảng confusion được điều kiện hóa theo coverage, cùng score tạo acceptance lồng nhau, cùng cơ chế ngăn selector loại quá nhiều mẫu thuộc lớp khó. Cần chứng minh phần tương tác giữa **cạnh nhầm lẫn × vị trí coverage × class acceptance budget**, không chỉ lợi ích của class weighting.

Group robustness trong selective classification đã có ở [Jones et al., ICLR 2021](#). Ràng buộc fairness trên coverage scale cũng có tiền lệ ở [Fair Selective Classification via Sufficiency, ICML 2021](#). [SelectiveNet, ICML 2019](#) đã tối ưu classification/rejection đồng thời với coverage constraint. Vì vậy novelty của CBR có rủi ro bị đánh giá incremental nếu chỉ khác ở cách ghép các thành phần quen thuộc.

Không có bảo đảm micro-AURC hoặc accuracy 100% tăng. Coverage floor có thể làm micro-risk tăng khi phải giữ nhiều ảnh thuộc lớp rất khó. Cần báo Pareto trade-off giữa micro performance và per-class coverage; nếu chỉ cải thiện fairness mà micro-AURC giảm chất lượng, hướng này chưa đạt đầy đủ mục tiêu cải thiện baseline theo yêu cầu ban đầu.

Soft constraint không phải bảo đảm coverage thật tại inference. Score có thể co lại khiến mask mềm gần hằng số c . Cần anneal T_s , kiểm tra ties/score variance và đánh giá hard top-k cùng hard per-class coverage. Không dùng soft feasibility để tuyên bố bảo đảm hữu hạn mẫu hay bảo đảm dưới distribution shift.

8.6. Ablation và điều kiện giữ hướng

So sánh SCSF với:

1. Class-weighted CE và class-balanced sampling đúng prior.
2. Group-DRO theo true class tại full coverage.
3. Multi-coverage global selective loss, chưa có confusion edges/floor.
4. Confusion objective, chưa có floor.
5. Coverage floor, chưa có confusion objective.
6. CBR đầy đủ; true-class group risk thay cho class-pair risk; một coverage thay cho nhiều coverage.

Báo micro-AURC, macro/per-class risk, coverage thấp nhất theo lớp, ma trận lỗi trên accepted set ở 90/95%, full accuracy và số lỗi tuyệt đối. Đo runtime/VRAM; phân thống kê thêm xấp xỉ $O(B|C|C + C^2|C|)$, chưa kể chi phí giải quantile mềm. Inference không thêm model/head.

Giữ hướng nếu: có cạnh nhầm lẫn chi phối accepted errors, phương pháp giảm chúng ổn định và không đạt kết quả bằng cách làm lớp đó mất coverage. Nếu floor làm micro-risk xấu đi trên hầu hết seeds, cần báo rõ trade-off hoặc dừng hướng cho mục tiêu hiện tại.

9. Ma trận novelty và đối chứng tối thiểu

Công trình/thành phần đã có	Phần dễ bị trùng	Điều cần chứng minh thêm trong đề xuất
ConfidNet, NeurIPS 2019	Confidence network dự đoán TCP	Không nhận việc thêm confidence head là mới
CRL, ICML 2020	Ranking confidence theo correctness	R3 cần giá trị của repairability routing vượt ranking thường
CCL-SC, ICML 2024	Học feature có confidence để cải thiện selective classification	Cần đối chứng SCSF+CCL-SC; “cải tiến feature cho SC” không còn là claim mới đủ cụ thể
TULIP, ICML 2024	Intermediate representations cho uncertainty	DTR cần state-conditioned repair, không chỉ lấy feature nhiều tầng
Reweighted Risk, v1 năm 2025	SoftRank và tối ưu AURC trong training	R3 cần vượt direct-AURC được cải đúng; lưu rõ phiên bản tham chiếu
ReSiDe, preprint 2026	Ghép confidence nhiều tầng bằng preference optimization	R3/DTR cần chứng minh gain từ thay đổi classifier/representation khi train
SDN, ICML 2019	Đúng ở tầng giữa, sai ở tầng cuối; disagreement	DTR cần vượt early-exit/disagreement và chứng minh repair final prediction

Công trình/thành phần đã có	Phần dễ bị trùng	Điều cần chứng minh thêm trong đề xuất
Student Self-reflection, IJCV 2023	Distillation shallow→deep	DTR cần tác dụng của điều kiện chuyển tiếp và RC weighting
Learning to Reweight, ICML 2018	Cập nhật ảo để suy ra trọng số mẫu	R3 không nhận virtual update/reweighting riêng lẻ là mới
Robust meta-learning for SC	Meta-objective huấn luyện auxiliary network	Cần đọc sâu khi định vị R3; “meta-learning cho SC” không phải claim mới
Fair SC via Sufficiency, ICML 2021	Giảm chênh lệch selective performance giữa nhóm	CBR cần vượt fairness/group methods, không chỉ thêm class constraint
Source train_ds_scsf.py hiện có	Multi-head, fusion, AURC surrogate, pairwise hinge	Đây là các ablation/đối chứng nội bộ phải chạy trước khi nhận novelty

Phạm vi rà soát: tìm tài liệu theo selective classification, risk-coverage/AURC training, correctness ranking, multi-depth uncertainty, repair/reject, shallow-to-deep distillation, group/fair selective classification, và kiểm tra các công trình gần nhất tới ngày review. Đã đọc đầy đủ baseline được cung cấp; mức đọc tài liệu liên quan gồm abstract, phần phương pháp phù hợp và các trang được dẫn, không phải một systematic review toàn bộ lĩnh vực.

Bản v1 của Zhou et al. mô tả SoftRank-AURC; **bản v5 cập nhật 29/01/2026** đã thay đổi trọng tâm/mô tả tối ưu với CDF approximation. Do đó tài liệu này dẫn **v1** cho tiền lệ SoftRank, không gán nguyên mô tả cũ cho phiên bản mới.

Không thấy công trình trùng chính xác cả cơ chế của từng đề xuất trong tập tài liệu đã kiểm tra. **Đó không phải chứng minh chưa từng tồn tại.** Trước khi chốt submission cần kiểm tra toàn văn, citations và các phiên bản mới của đối chứng gần nhất, đặc biệt cho R3 và CBR.

10. Protocol thực nghiệm để chứng minh cải tiến là thật

10.1. Khóa baseline trước

Thiết lập một cấu hình tham chiếu bám PDF: correctness-NLL, feature gradient mở, logits vào meta-head detach, $\alpha_e = 1$, cosine decay tới 10^{-4} , cùng backbone/preprocessing/split. Trong script hiện có, các cờ liên quan là `--brier --nll --meta-weight-mode decay --init-meta-weight 1.0 --min-meta-weight 0.0001 --error-weight 1.0`, **không bật --end-to-end** để giữ logit detach.

Đây là bản đồ cấu hình, **chưa phải lệnh đã được chạy hoặc bản tái lập đã xác nhận.** Những chi tiết như epoch/warmup, dataset split, kiến trúc checkpoint và supplementary settings cần được ghi thành config đầy đủ trước thí nghiệm. Các sửa lỗi loss/evaluator và cleanup cấu hình áp dụng như nhau cho baseline và phương pháp mới, không tính vào contribution.

10.2. Chia dữ liệu và ngân sách công bằng

- Tách train/validation/test cố định. Chọn checkpoint và hyperparameters chỉ bằng validation. Mọi dữ liệu dùng gradient, virtual update hay train probe phải được coi là train.
- Nếu giữ protocol cũ lấy validation từ official test, lưu đúng sample IDs và chỉ so với baseline chạy cùng protocol; không so trực tiếp với số trên official test đầy đủ.
- Với ảnh y khoa có nhiều ảnh/lát cắt mỗi bệnh nhân, split theo patient nếu metadata cho phép. Chưa có metadata thì ghi rõ chưa kiểm chứng patient independence; không tuyên bố đã xử lý leakage.
- Bắt đầu 3 training seeds độc lập cho screening; hướng được chọn chạy ít nhất 5 seeds nếu ngân sách cho phép. Cùng seed/split giữa các phương pháp để so paired.
- Mỗi hướng dùng cùng backbone, init/pretrain, augmentation và số update ở đối chứng tương ứng. Khi phương pháp thêm compute, có thêm đối chứng baseline train lâu hơn hoặc cùng wall-clock; vẫn báo cả kết quả cùng epochs và cùng compute.
- Screening cấu hình mới trên validation, rồi khóa cấu hình trước test. Không lấy mỗi coverage từ một checkpoint khác nhau để ghép thành một RC curve.

10.3. Bộ dữ liệu theo thứ tự đầu tư

1. **CIFAR-10/VGG16-BN**: kiểm tra tái lập, chẩn đoán lỗi ở coverage 90/95%, kiểm tra tất cả ablations rõ.
2. **Một dataset y khoa khó mà SCSF đang thắng**, như Breast Ultrasound hoặc Chest CT: còn nhiều lỗi để phân tích và tránh chỉ chọn nơi baseline yếu.
3. **Một dataset SCSF đang thua**, như Alzheimer hoặc Brain Tumor MRI: kiểm tra mức khái quát, báo cả số lỗi tuyệt đối vì AURC có thể rất nhỏ.
4. **ResNet-18 và thêm một dataset khác** cho hướng cuối cùng; nếu công bố rộng hơn, mở rộng số dataset/architecture phù hợp claim.

Không dùng ngay tất cả tám dataset để tune hàng loạt. Với hướng 2, đo trước các transition rates; với hướng 3, đo trước per-class coverage và accepted confusion matrix để kiểm tra giả thuyết nền.

10.4. AURC, E-AURC và threshold phải nhất quán

Dùng định nghĩa $AURC_n$ ở §5 với toàn bộ n prefix. Oracle empirical giữ nguyên số lỗi của **chính classifier đang đánh giá**, xếp mọi mẫu đúng trước mọi mẫu sai:

$$E-AURC_n = AURC_n - AURC_n^{oracle}.$$

E-AURC bổ sung thông tin ranking, không thay accuracy và AURC. Không dùng một oracle chung cho các classifier có số lỗi khác nhau. Nếu cần so đúng con số paper theo quy ước trapezoid cũ, xuất thêm cột riêng với cùng evaluator cho mọi phương pháp; không trộn hai quy ước.

Quy tắc khác:

- Risk fraction và risk % phải phân biệt rõ; ví dụ AURC 0.00059 tương đương AURC 0.059% trong Bảng 3.
- Full curve để so ranking có thể lấy top-k trên test scores mà không dùng label chọn threshold. Khi báo một operating threshold dùng triển khai, chọn h trên validation và báo cả **coverage thực đạt** trên test.
- $\text{coverage@risk} \leq r$ lấy từ test curve là metric mô tả có dùng test labels, không phải threshold được phép chọn rồi tuyên bố bảo đảm ngoài mẫu. Ghi tách biệt hai kiểu báo cáo.
- Đặt tie-breaking độc lập correctness/label, cố định giữa các phương pháp; báo tỷ lệ ties nếu sigmoid bị bão hòa.
- Partial AURC [0.8,1] ghi rõ có chia cho độ rộng 0.2 hay không. Đề xuất báo bản chuẩn hóa $\frac{1}{0.2} \int_{0.8}^1 R(c)dc$.

10.5. Phân biệt gain classifier, gain score và gain cơ chế

Với mỗi trained backbone, đánh giá cả score riêng và SR. Thêm đối chứng freeze feature khi train confidence head. Mục tiêu là phân biệt:

Kết quả quan sát	Claim được hỗ trợ
Accuracy@100 tăng, AURC giảm, SR cũng tốt hơn	Backbone có cải thiện; chưa đủ kết luận score mới tốt hơn
Accuracy@100 gần như giữ nguyên, E-AURC giảm	Có bằng chứng ranking tốt hơn
Chỉ ECE tốt hơn, RC không tốt hơn	Chưa đạt mục tiêu đề bài
Micro-AURC giảm nhưng lớp khó gần như bị reject hết	Cần báo trade-off; chưa đủ cho claim CBR
Chỉ train score phân tách tốt, validation không tốt	Overfit confidence, chưa có gain tổng quát hóa
Chỉ tốt hơn script surrogate cài sai	Chứng minh sửa implementation, chưa chứng minh contribution mới

Báo mean/std qua **training seeds**, paired confidence interval của chênh lệch chính, và bootstrap theo patient khi có cấu trúc bệnh nhân. Sample bootstrap và training-seed variance là hai nguồn biến thiên khác nhau; không dùng một loại thay hoàn toàn cho loại kia.

Tiêu chí thành công đăng ký trước: giảm AURC và risk tại vùng coverage mục tiêu trên held-out data, lặp lại qua seeds/datasets, trong khi full accuracy không suy giảm vượt biên non-inferiority được chọn trước trên validation. Không chọn biên sau khi thấy test. Nếu chỉ một vùng coverage tăng, mô tả đúng vùng đó thay vì nói “thống trị toàn bộ RC”.

Không đặt lời hứa “+2% accuracy” hoặc “−20% AURC” cho ba phương pháp trước khi chạy. Số baseline là mốc đối chiếu, không phải bằng chứng phương pháp mới sẽ vượt được.

11. Điểm thay đổi dự kiến trong source khi triển khai

Tài liệu này **chưa sửa code training hoặc chạy training**. Các thay đổi dưới đây là bản đồ triển khai để review phương pháp.

Vị trí hiện có	R3	DTR	CBR
<code>train_scsf.py::MetaCalibrator</code>	Cho phép lấy raw score logit để rank ổn định	Đổi output thành 4 states, trả marginal final-correctness	Cho phép lấy raw score cho quantile mềm
<code>VGG16BN_FeatureExtractor</code>	Giữ feature gradient; hỗ trợ functional virtual step ở block cuối	Thêm probe train tại pool4; dùng feature hiện có	Giữ kiến trúc
<code>train_epoch_joint</code>	Tính ranks, virtual repairability, rank/repair objectives	Tính transition labels, conditional KD, joint NLL	Tính soft masks, confusion risks, class floors và dual updates
Data loader	Sample ID nếu dùng cache; hai view train	Hai view train kiểm tra teacher ổn định	Kiểm soát class support và prior của sampler
Evaluation	Chung một evaluator, lưu predictions/scores	Thêm transition diagnostics, probe tùy chọn khi phân tích	Thêm per-class coverage và confusion trên accepted set

Nên tách các loss mới thành module riêng khi thực hiện để ablation rõ ràng. `train_ds_scsf.py` là nguồn đối chứng multi-head sẵn có, không nên dùng nguyên các comment trong đó thay cho specification toán học.

12. Đề xuất chốt hướng sau vòng thí nghiệm đầu

Ưu tiên DTR-SCSF. Đầu tiên kiểm tra lỗi đúng→sai ở depth trên validation. Nếu tỷ trọng đủ lớn, triển khai auxiliary CE → conditional KD → joint transition confidence → RC weighting, theo đúng thứ tự ablation. Một đóng góp tốt có thể trình bày là:

Chúng tôi xác định các chuyển tiếp dự đoán đúng ở tầng giữa nhưng sai ở tầng cuối là một nguồn gây lỗi được nhận trong selective classification; từ đó đề xuất training có giám sát chuyển tiếp và sửa lỗi theo tác động lên risk-coverage.

Đây là **claim dự kiến**, chỉ dùng như kết luận nghiên cứu sau khi có bằng chứng tương ứng. Nếu kiểm tra cơ chế không đạt, không giữ claim bằng cách chỉ trình bày một bảng metric đẹp.

Ưu tiên thứ hai R3-SCSF khi lỗi confidence cao còn nhiều và virtual repairability có predictive value vượt proxy rẻ. **CBR-SCSF** phù hợp nếu phân tích lớp cho thấy baseline đạt RC tốt bằng acceptance lệch mạnh; cần chấp nhận khả năng kết quả chính là trade-off thay vì tăng micro-accuracy.

Ba hướng đều thay đổi supervision/objective hoặc gradient của backbone **trong training**. Thresholding ở inference là cơ chế selective classification sẵn có, không phải phần được tính là cải tiến mới.